



Gymnázium J. K. Tyla

Agentic AI, Agentic Engineering and Harness Engineering

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

Rozsah práce: 2871 slov / 19058 znaků
Rozsah práce: 2784 slov / 18505 znaků

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

Tato odborná práce se zabývá současným využitím agentní umělé inteligence při vývoji softwaru. Popisuje, jaké části vývojového procesu mohou dnešní agentní systémy provádět samostatně a jaké nástroje, pravidla a kontrolní mechanismy potřebují pro opakovatelnou práci nad skutečným repozitářem. Praktickou část tvoří návrh a implementace systému DarkFactory, který tyto principy převádí do konkrétní architektury agentního harnessu.

Abstract

This thesis examines the current use of agentic artificial intelligence in software development. It describes which parts of the development process today's agentic systems can perform independently and which tools, rules, and control mechanisms they need for repeatable work on a real repository. The practical part is the design and implementation of DarkFactory, which applies these principles in a concrete agent-harness architecture.

Keywords (Klíčová slova)

Agent Harness (Agentní harness), Agent Loop (Smyčka ReAct) [ReAct Loop], Agentic Engineering (Agentické inženýrství), CI (Průběžná integrace) [Continuous Integration], Context Engineering (Kontextové inženýrství), Guardrail (Deterministický mantinel) [Deterministic Guardrail], HITL (Zapojení člověka do smyčky) [Human-in-the-loop], LLM (Jazykový model) [Large Language Model], Prompt Engineering (Promptové inženýrství), Sandbox (Izolované běhové prostředí), Tools (Nástroje), Version Control (Správa verzí)

Obsah

<u>Prohlášení</u>	2
<u>Anotace</u>	3
<u>Abstract</u>	4
<u>Keywords (Klíčová slova)</u>	5
1 Úvod	9
1.1 Motivace a vymezení problému	9
1.2 Cíl práce a výzkumné otázky	10
1.2.1 Hlavní cíl	11
1.2.2 Dílčí cíle	11
1.2.3 Výzkumné otázky	11
1.3 Metodika práce	11
2 Teoretická část	12
2.1 Úvod	12
2.2 Software Engineering (Softwarové inženýrství)	12
2.2.1 Vibe Coding	12
2.2.2 Slop	13
2.2.3 Spec-Driven Development (Vývoj řízený specifikací)	13
2.2.4 Plánování	13
2.2.5 Version Control (Správa verzí)	13
2.2.6 GitHub	14
2.2.7 Runtime (Běhové prostředí)	14
2.2.8 CI (Průběžná integrace) [Continuous Integration]	14
2.2.9 GitHub Actions	14
2.2.10 Container (Softwarový kontejner) [Software Container]	14
2.2.11 Integration Test (Integrační test)	14
2.3 LLM (Jazykový model) [Large Language Model]	14
2.3.1 Transformer	15
2.3.2 Tokenizér	15
2.3.3 Token	15
2.3.4 Embedding (Vektorová reprezentace)	15
2.3.5 Kontextové okno	16
2.3.6 KV Cache (Mezipaměť klíčů a hodnot) [Key-Value Cache]	16
2.3.7 Degradace kontextu	16
2.4 Harness	16

2.4.1	Turn (Tah interakce)	18
2.4.2	Session (Agentní sezení) [Agent Session]	18
2.4.3	Transcript (Přepis)	18
2.4.4	Agent Loop (Smyčka ReAct) [ReAct Loop]	19
2.4.5	Patologie divergence	19
2.4.6	Plugins (Rozšíření)	19
2.4.7	Subagent (Podřízený agent)	19
2.4.8	Tools (Nástroje)	20
	JSON Schema Tool Calling (Vyvolávání nástrojů s JSON Schema)	20
	Code Execution (Spouštění kódu)	20
	MCP (Model Context Protocol)	20
2.4.9	Skills (Dovednosti)	20
2.4.10	Scripts (Skripty)	21
2.4.11	Hooks (Událostní zachytné body)	21
2.5	Agentic Engineering (Agentické inženýrství)	21
2.5.1	Guardrail (Deterministický mantinel) [Deterministic Guardrail]	22
2.5.2	HITL (Zapojení člověka do smyčky) [Human-in-the-loop]	22
2.5.3	Sandbox (Izolované běhové prostředí)	22
2.5.4	Harness Engineering (Harnessové inženýrství)	22
2.5.5	Prompt Engineering (Promptové inženýrství)	22
	System Prompt (Systémový prompt)	22
2.5.6	Loop Engineering (Inženýrství prováděcí smyčky) [Execution-loop Engineering]	23
2.5.7	Graph Engineering (Inženýrství pracovních grafů) [Workflow-graph Engineering]	23
	DAG (Orientovaný acyklický graf) [Directed Acyclic Graph]	23
2.5.8	Context Engineering (Kontextové inženýrství)	23
	Context Injection (Vkládání kontextu)	23
	Compaction (Kompakce kontextu) [Context Compaction]	23
	RAG (Generování rozšířené vyhledáváním) [Retrieval-Augmented Generation]	24
	Sémantický posun	24
3	Praktická část	25
3.1	Úvod	25
3.2	Architektura DarkFactory	25
3.3	Výsledky a diskuse	25

3.3.1	Vyhodnocení výzkumných otázek	25
3.3.2	Omezení evaluace	26
4	Závěr	27
	<u>Seznam obrázků a tabulek</u>	28
	<u>Seznam příloh</u>	29
	Seznam zdrojů	30

1 Úvod

Tato práce zkoumá, jak dnes účinně využívat agentní umělou inteligenci při vývoji softwaru.

Zaměřuje se na to, co současní agenti dokážou samostatně provést a jaké prostředí, nástroje a pravidla potřebují, aby jejich práce byla opakovatelná a kontrolovatelná.

Praktickým příkladem je DarkFactory, na kterém jsou popsány principy převedeny do konkrétní architektury a vývojového procesu. Cílem úvodu je stručně vymezit, co práce zkoumá, proč je téma aktuální a podle čeho bude navržené řešení posuzováno.

1.1 Motivace a vymezení problému

Ukázat, jaké úlohy dokážou současné agentní systémy samostatně provádět a jaká technická vrstva umožňuje převést schopnost jazykového modelu do spolehlivého jednání nad skutečným softwarovým projektem.

Generativní AI se během několika let rozšířila do masového používání a současně se rozšiřují systémy, které plánují více kroků, pracují se soubory a nástroji, spouštějí příkazy a testy a vykonávají delší úlohy.

V softwarovém inženýrství tento posun reprezentují například Codex a Claude Code, které pracují nad repozitáři, upravují soubory a spouštějí testy. ^(1, 2)

 **Metodické vymezení / Rozsah práce:** Údaj 2,4 miliardy / 29 % je přibližný sekundární odhad. V práci má sloužit pouze jako kontext motivace, nikoli jako přesná empirická metrika.

CPA.RIP dne 14. září 2026 publikoval odhad Gradually AI, podle kterého generativní AI používá přibližně 2,4 miliardy lidí, tedy 29 % světové populace. Výpočet vychází z dat DataReportal pro duben 2026 a článek výslovně uvádí, že všechny globální hodnoty jsou přibližné. ⁽³⁾

How many people use AI? The world population as a dot grid

Each dot represents roughly 3.3 million people

2,500 dots = 8.3 billion people. Color = most advanced AI usage. Own estimate, as of August 2026.



Sources: DataReportal, OpenAI, ITU et al.

gradually.ai

Obrázek 1: Odhad používání AI ve světové populaci. Zdroj: CPA.RIP / Gradually AI ⁽³⁾.

1.2 Cíl práce a výzkumné otázky

Tato část stanovuje hlavní cíl, dílčí cíle a výzkumné otázky práce. Společně vymezují rozsah návrhu DarkFactory a rámec jeho evaluace.

1.2.1 Hlavní cíl

Zjistit, jak lze současnou agentní AI účinně používat při vývoji softwaru, a navrhnout architekturu agent harnessu, která podporuje vysokou míru autonomie při zachování lidského dohledu v důležitých rozhodnutích.

DarkFactory slouží jako konkrétní návrh této architektury.

1.2.2 Dílčí cíle

- Vymezit infrastrukturu pro správu verzí a průběžné automatické ověřování změn.
- Popsat limity modelového kontextu, jeho správu a mechanismy pro dlouhotrvající agentní úlohy.
- Popsat nástrojové a prováděcí mechanismy současného agentního harnessu.
- Navrhnout způsob, jak zachovat lidský dohled nad důležitými rozhodnutími bez nutnosti ručně provádět každou rutinní změnu.

Dílčí cíle pokrývají vývojové prostředí, kontext, provádění a lidský dohled.

1.2.3 Výzkumné otázky

- O1: Jak lze současný agentní systém zapojit do vývoje softwaru tak, aby samostatně prováděl co největší část běžné práce a člověk se soustředil na zadání, kontrolu a důležitá rozhodnutí?
- O2: Jaké mechanismy harnessu pomáhají rozpoznat a zastavit neproduktivní opakování, oscilaci nebo zacyklení během delší úlohy?
- O3: Jak spravovat pracovní kontext agenta tak, aby při delších úlohách neztrácel důležité požadavky a stav projektu?

O1–O3 pokrývají autonomii, stabilitu prováděcí smyčky a správu pracovního kontextu.

1.3 Metodika práce

Práce porovnává veřejně popsané principy současných agentních systémů, rozděluje je do samostatných konceptů a používá je při návrhu DarkFactory.

Předmětem práce není trénování neuronových sítí, optimalizace vah ani podrobná matematika modelového učení. *LLM (Jazykový model) [Large Language Model]* je chápán jako hotová inferenční komponenta a je popsán pouze v rozsahu potřebném pro další části práce.

Architektura kolem modelu je rozložena do samostatných konceptů, aby měl každý mechanismus vlastní definici, popis a nebylo nutné stejné vysvětlení opakovat v několika kapitolách.

Navržené principy jsou následně promítnuty do DarkFactory.

2 Teoretická část

Vymezení konceptů potřebných pro pochopení současného agentního vývoje softwaru. Postupuje od softwarového inženýrství přes jazykový model a harness k agentickému inženýrství.

2.1 Úvod

Vymezení hranic teoretické části a vztahů mezi jejími hlavními vrstvami. Jazykový model je v práci chápán jako hotová inferenční komponenta. Teoretická část postupuje přes Software Engineering, LLM a Harness k Agentic Engineering, pod které spadá také Harness Engineering.

2.2 Software Engineering (Softwarové inženýrství)

Systematické uplatňování inženýrských principů na specifikaci, vývoj, ověřování, provoz a údržbu softwarových systémů. ⁽⁴⁾

Pro tuto práci poskytuje procesní rámec, ve kterém musí být změny nejen vytvořeny, ale také řízeny, ověřeny a integrovány.

2.2.1 Vibe Coding

Způsob tvorby softwaru, při kterém člověk iteruje pomocí pokynů v přirozeném jazyce bez průběžné kontroly vygenerovaného kódu. ⁽⁵⁾

Termín zavedl Andrej Karpathy v roce 2025 pro styl práce, ve kterém vývojář převážně přijímá a směřuje modelový výstup místo detailního čtení kódu. ^(5,6)

Tweet Andreje Karpathyho z 2. února 2025, ve kterém popsal původní význam Vibe Coding.



Andrej Karpathy ✓
@karpathy



There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

3:17 PM · Feb 2, 2025 · 6.8M Views

Obrázek 2: Původní tweet Andreje Karpathyho o Vibe Coding. ^(5,7)

2.2.2 Slop

Neformální označení pro nekvalitní digitální obsah, zejména obsah vytvořený umělou inteligencí. ⁽⁸⁾

V této práci označuje zejména AI-generovaný software, jehož objem nebo zdánlivá úplnost převyšují jeho ověřenou funkčnost a udržitelnost.

2.2.3 Spec-Driven Development (Vývoj řízený specifikací)

Přístup k AI-asistovanému vývoji, ve kterém explicitní specifikace řídí plánování, implementaci a ověřování změny. ⁽⁹⁾

Specifikace odděluje požadované chování a omezení od konkrétní implementace a slouží jako společný referenční bod pro člověka i agenta. ⁽⁹⁾

2.2.4 Plánování

Převod požadavku na explicitní kroky, závislosti a podmínky ověření před prováděním změn.

Plán rozděluje práci na kontrolovatelné části a určuje, podle jakých podmínek lze posoudit jejich dokončení. ⁽⁴⁾

2.2.5 Version Control (Správa verzí)

Systém pro zaznamenávání historie změn souborů a práci s oddělenými liniemi vývoje.

⁽¹⁰⁾

Historie commitů umožňuje změny porovnávat, vracet a slučovat; větve umožňují oddělit souběžnou práci před integrací. ⁽¹⁰⁾

2.2.6 GitHub

Platforma pro hostování gitových repozitářů a spolupráci nad změnami prostřednictvím větví a pull requestů. ^(11, 12)

Pull request soustřeďuje navrženou změnu, diskusi, revizi, automatické kontroly a následné sloučení do cílové větve. ⁽¹²⁾

2.2.7 Runtime (Běhové prostředí)

Prostředí a systémové prostředky dostupné programu během jeho vykonávání. Běhové prostředí určuje například dostupné procesy, souborový systém, síť a oprávnění; jeho hranice lze dále omezit kontejnerem nebo sandboxem. ⁽¹³⁾

2.2.8 CI (Průběžná integrace) [Continuous Integration]

Vývojová praxe, při níž se změny průběžně integrují a automaticky ověřují sestavením, testy a dalšími kontrolami. ⁽¹⁴⁾

CI převádí část podmínek kvality do opakovatelných strojově vyhodnotitelných kontrol spouštěných nad změnami. ⁽¹⁴⁾

2.2.9 GitHub Actions

Automatizační platforma GitHubu, která spouští workflow složená z jobů a kroků v reakci na události, plán nebo ruční spuštění. ⁽¹⁵⁾

Workflow jsou verzované YAML soubory v repozitáři a mohou automatizovat build, testování i nasazení. ⁽¹⁵⁾

2.2.10 Container (Softwarový kontejner) [Software Container]

Izolovaný proces s vlastním uživatelským prostředím a soubory potřebnými pro běh aplikace, který sdílí jádro hostitelského operačního systému. ⁽¹⁶⁾

Kontejner umožňuje balit aplikaci a její závislosti do přenositelného běhového prostředí, ale neposkytuje stejnou izolační hranici jako samostatný virtuální stroj. ⁽¹⁶⁾

2.2.11 Integration Test (Integrační test)

Ověření spolupráce více komponent nebo vrstev systému přes jejich rozhraní. ⁽⁴⁾
Integrační test zachycuje chyby vznikající ve vzájemném propojení částí systému, které izolované testování jednotlivých komponent nemusí odhalit. ⁽⁴⁾

2.3 LLM (Jazykový model) [Large Language Model]

Neuronový model pro zpracování a generování posloupností tokenů. Současné LLM typicky používají architekturu *Transformer*, která při autoregresivním generování odhaduje další token z předchozí sekvence. ⁽¹⁷⁾ Práci s nástroji, stavem a prostředím zajišťuje okolní agentní runtime nebo harness, nikoli samotná textová inference. ⁽¹⁸⁾

2.3.1 Transformer

Architektura neuronové sítě založená na mechanismu pozornosti, který modeluje vztahy mezi prvky sekvence. ⁽¹⁷⁾

Při autoregresivním generování dekodér z dosavadní sekvence vytváří reprezentaci použitou k odhadu následujícího tokenu. ⁽¹⁷⁾

2.3.2 Tokenizér

Komponenta, která převádí vstup na posloupnost tokenů a jejich identifikátorů a provádí odpovídající zpětné dekodování.

Pravidla tokenizace určují slovník a segmentaci vstupu; subword metody, například Byte Pair Encoding, umožňují skládat text z jednotek menších než celé slovo. ⁽¹⁹⁾

2.3.3 Token

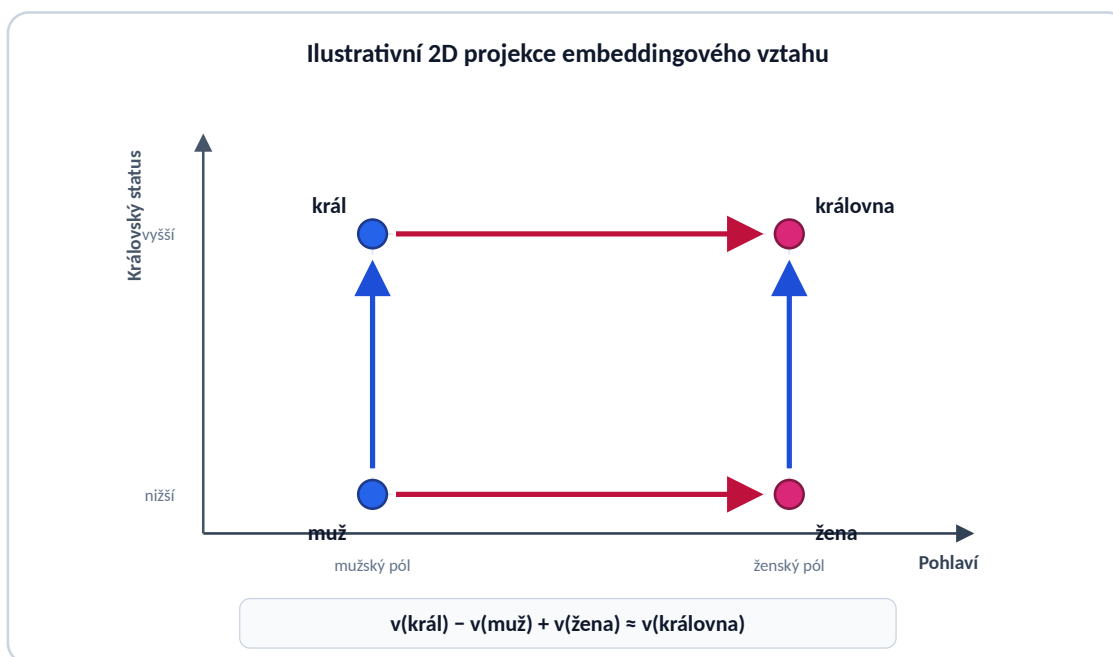
Diskrétní jednotka vstupní nebo výstupní sekvence reprezentovaná identifikátorem ve slovníku tokenizéru.

Token nemusí odpovídat celému slovu; subword tokenizace může pracovat s částmi slov nebo jinými textovými fragmenty. ⁽¹⁹⁾ Identifikátor tokenu je před zpracováním neuronovou sítí převeden na vektorovou reprezentaci.

2.3.4 Embedding (Vektorová reprezentace)

Vícerozměrná vektorová reprezentace diskretních prvků, v níž geometrické vztahy mohou zachycovat užitečné vztahy mezi reprezentovanými objekty. ⁽²⁰⁾

V jazykovém modelu embedding převádí identifikátory tokenů na spojitě vektory zpracovatelné neuronovou sítí; vektorové reprezentace lze také porovnávat podle podobnosti. ⁽²⁰⁾



Obrázek 3: Ilustrativní 2D projekce vztahu $\text{král} - \text{muž} + \text{žena} \approx \text{královna}$; osy Pohlaví a Královský status slouží pouze k názornému vysvětlení a nepředstavují doslovné naučené dimenze embeddingového prostoru.

2.3.5 Kontextové okno

Maximální rozsah tokenové sekvence dostupný modelu v jednom inferenčním běhu. Do aktivního kontextu se společně vkládají instrukce, uživatelské vstupy, historie a výsledky nástrojů. Samotná nominální délka okna nezaručuje, že model všechny vložené informace využije stejně spolehlivě. ⁽²¹⁾

2.3.6 KV Cache (Mezipaměť klíčů a hodnot) [Key-Value Cache]

Mezipaměť dříve vypočtených klíčů a hodnot v pozornostních vrstvách transformeru používaná při autoregresivním generování. Při generování dalšího tokenu lze uložené klíče a hodnoty předchozí sekvence znovu použít místo jejich úplného přepočítání; paměťové nároky cache přitom rostou s délkou aktivní sekvence. ^(22, 23)

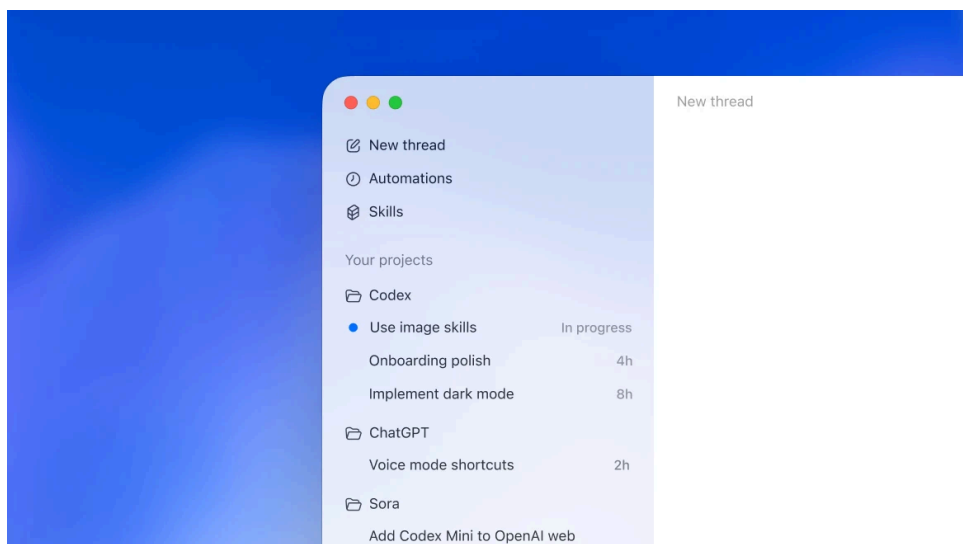
2.3.7 Degradace kontextu

Pokles spolehlivosti, s níž model využívá relevantní informace v dlouhém nebo zahlceném kontextu. Experimenty s dlouhým kontextem ukazují, že výkon může záviset na poloze relevantní informace a klesat, když je umístěna uvnitř dlouhého vstupu. ⁽²¹⁾

2.4 Harness

Běhová a orchestrační vrstva, která propojuje jazykový model s nástroji, stavem a prostředím. Harness spravuje pracovní kontext, provádění nástrojů, stav běhu a podmínky pokračování nebo ukončení; opakované jednání koordinuje **Agent Loop (Smyčka ReAct) [ReAct Loop]**. ⁽¹⁸⁾

Codex. Agentní vývojový systém OpenAI určený pro samostatné plnění softwarově-inženýrských úloh. Desktopová aplikace umožňuje řídit více agentů paralelně, oddělovat jejich práci do vláken a worktree, kontrolovat diffy a delegovat dlouhotrvající úlohy. Rozhraní pro práci s agentními úlohami a dovednostmi.



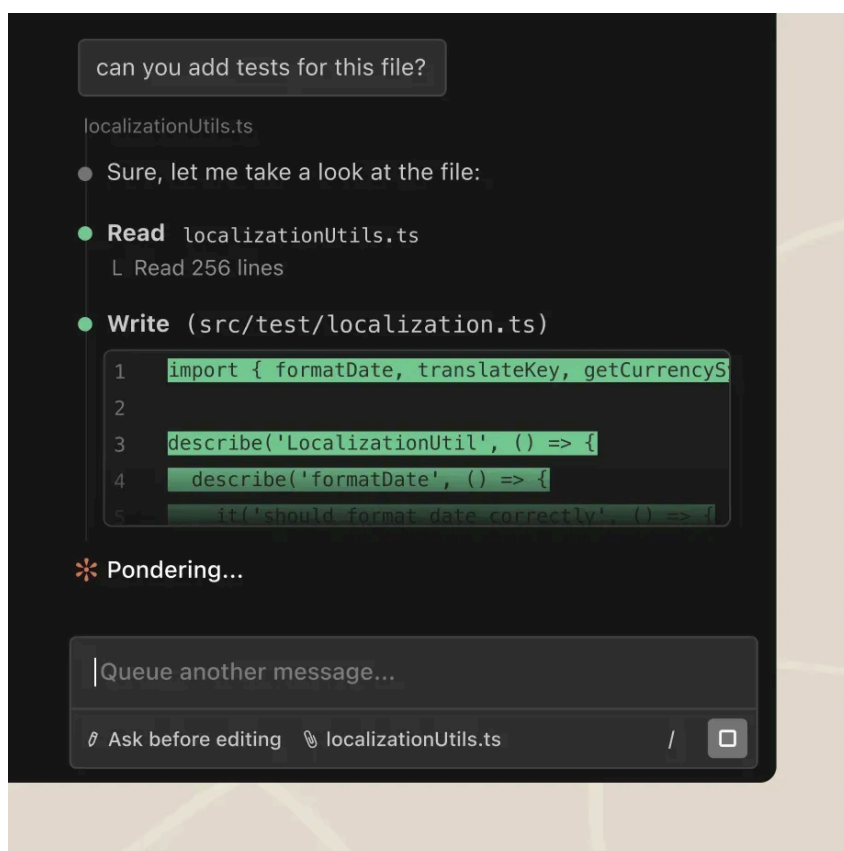
Obrázek 4: Aplikace Codex. Zdroj: OpenAI.

(1, 1)

Claude Code. Agentní vývojové prostředí společnosti Anthropic dostupné v terminálu, IDE, na webu a v desktopové aplikaci.

Claude Code může plánovat změny, upravovat soubory, spouštět příkazy a testy, používat Git a MCP a vytvářet pull requesty.

Čtení souboru, zápis změny a průběžný stav agentní úlohy.



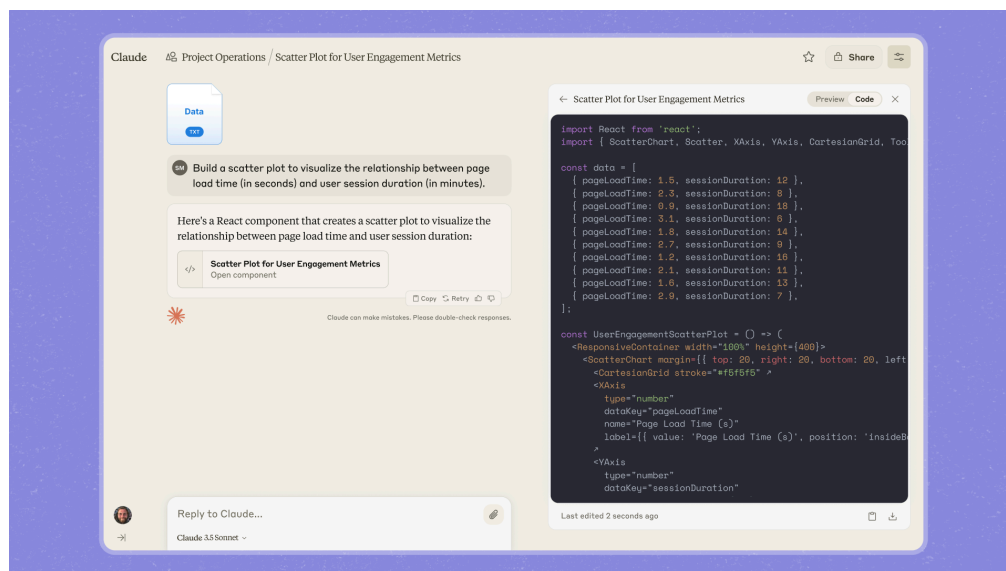
Obrázek 5: Claude Code při práci se souborem. Zdroj: Anthropic.

(2, 2)

Claude Desktop (Aplikace Claude pro desktop) [Claude Desktop App]. Desktopová aplikační vrstva nad modely Claude, která propojuje konverzaci s lokálními soubory, aplikacemi, webem a dalšími schopnostmi.

Se souhlasem uživatele může aplikace číst, upravovat a ukládat soubory a pracovat s dalšími aplikacemi.

Konverzace s pracovním artefaktem a samostatným pracovním povrchem.



Obrázek 6: Aplikační rozhraní Claude. Zdroj: Anthropic.

(24, 24)

2.4.1 Turn (Tah interakce)

Jedna diskretní jednotka interakce v konverzačním nebo agentním běhu, například vstup uživatele, výstup modelu nebo výsledek nástroje.

Posloupnost tahů tvoří historii, ze které harness sestavuje vstup pro další iteraci agentní smyčky. ⁽²⁵⁾

2.4.2 Session (Agentní sezení) [Agent Session]

Ohraničený persistovaný stav interakce, který umožňuje navazovat na předchozí průběh agentního běhu.

Session ukládá historii interakce a při dalším běhu ji může znovu načíst do pracovního kontextu. ⁽²⁶⁾

2.4.3 Transcript (Přepis)

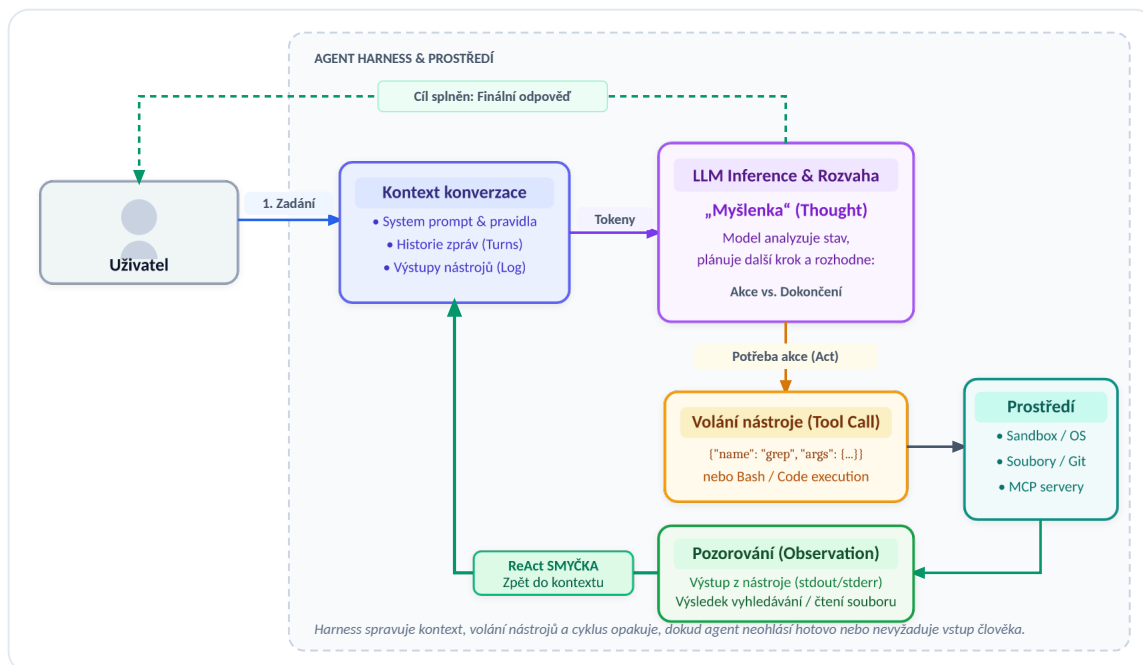
Uspořádaný záznam položek vzniklých během session, například zpráv, tahů a výsledků nástrojů.

Persistovaná historie umožňuje rekonstruovat předchozí průběh a vybírat informace pro pokračování dalšího běhu. ⁽²⁶⁾

2.4.4 Agent Loop (Smyčka ReAct) [ReAct Loop]

Iterativní cyklus, v němž model vyhodnotí stav, zvolí akci, harness ji provede a výsledek vrátí do další iterace. ⁽²⁵⁾

ReAct formalizuje střídání rozhodnutí, akce a pozorování výsledku; nové pozorování se stává vstupem dalšího kroku. ⁽²⁵⁾



Obrázek 7: Smyčka ReAct: model zvolí akci, harness ji provede a výsledek vrátí modelu.

2.4.5 Patologie divergence

Selhání agentní smyčky, při němž další iterace nepřibližují běh k cíli. Projevem může být opakování neúčinných kroků nebo pokračování bez měřitelného pokroku; mechanismy reflexe a zpětné vazby jsou jedním ze způsobů, jak takové opakované selhání omezovat. ⁽²⁷⁾

2.4.6 Plugins (Rozšíření)

Distribuovatelný balíček rozšíření, kterým lze doplnit schopnosti a běhové chování agentního prostředí. Plugin může sdružovat více komponent, například skills, agenty, hooks a konfiguraci MCP serverů. ⁽²⁸⁾

2.4.7 Subagent (Podřízený agent)

Specializovaná agentní instance, které nadřazený orchestrátor deleguje vymezenou dílčí úlohu. Rozdělení práce mezi více agentních instancí umožňuje oddělit role a dílčí kontexty a následně předat výsledek zpět koordinujícímu procesu. ⁽²⁹⁾

2.4.8 Tools (Nástroje)

Rozhraní, kterým agent vyvolává operace mimo samotnou textovou inferenci, například čtení dat, volání API nebo změnu stavu systému.

Harness zprostředkuje požadavek na nástroj, provede operaci v prostředí a vrátí její výsledek modelu jako další pozorování. ^(18, 30)

JSON Schema Tool Calling (Vyvolávání nástrojů s JSON Schema)

Popis parametrů nástroje pomocí JSON Schema, podle kterého lze validovat strukturu vygenerovaných argumentů. ⁽³¹⁾

Schéma určuje například povinná pole, datové typy a další omezení; structured-output mechanismy mohou generování argumentů omezit tak, aby zadanému schématu odpovídaly. ⁽³²⁾

Code Execution (Spouštění kódu)

Nástrojová schopnost umožňující vykonat program nebo příkaz a vrátit jeho skutečný výstup modelu. ⁽³³⁾

Při vývoji softwaru zpřístupňuje agentovi například testy, buildy, formátovače a diagnostické příkazy místo odhadování jejich výsledku. ⁽³³⁾

MCP (Model Context Protocol)

Otevřený protokol pro standardizované propojení AI aplikací s externími nástroji a datovými zdroji. ⁽³⁴⁾

MCP odděluje klientskou AI aplikaci od serverů poskytujících nástroje a další schopnosti, takže integrace lze implementovat mimo vlastní jádro harnessu. ^(34, 35)

2.4.9 Skills (Dovednosti)

Znovupoužitelný balíček instrukcí a volitelných zdrojů, který se načítá pro úlohy odpovídající jeho účelu. ⁽³⁶⁾

Agent Skill je definován souborem `SKILL.md` a může odkazovat na doplňující skripty, reference nebo další zdroje načítané podle potřeby. ⁽³⁶⁾

SKILL.md (Formát SKILL.md) [SKILL.md Format]. Povinný definiční soubor Agent Skill.

Podle specifikace obsahuje YAML frontmatter následovaný instrukcemi v Markdownu; povinnými poli frontmatteru jsou `name` a `description`. ⁽³⁶⁾

Následující úplný příklad používá povinná pole i několik volitelných polí povolených specifikací: ⁽³⁶⁾

```
---
name: repository-tests
description: Run, diagnose, and summarize the repository test suite. Use
when a software change must be validated before completion.
license: MIT
compatibility: Requires the repository's test runner to be available in
```

```

the runtime.
metadata:
  author: DarkFactory
  version: "1.0"
allowed-tools: Bash Read
---

# Repository tests

Use this skill when a change must be validated against the repository's
tests.

## Procedure

1. Detect the test command documented by the repository.
2. Run the smallest relevant test scope first.
3. Fix or report failures before continuing.
4. Run the complete required suite before completion.
5. Report the commands run and the final result.

## Failure reporting

For every unresolved failure, include the command, failing target, and
relevant output.

```

Specifikace dovoluje vedle `SKILL.md` také volitelné adresáře například pro skripty, reference a assety; ty se načítají pouze podle potřeby. ⁽³⁶⁾

2.4.10 Scripts (Skripty)

Spustitelné soubory nebo posloupnosti příkazů používané k deterministickému provedení opakovatelné operace.

Agentní rozšíření mohou skripty používat pro transformace, validace nebo jiné kroky, které je výhodnější provést programově než novým modelovým rozhodnutím. ^(28, 36)

2.4.11 Hooks (Událostní záchytné body)

Definované body životního cyklu nebo události, na které lze navázat vlastní deterministickou logiku před, po nebo místo standardního chování systému.

Používají se například před spuštěním nástroje, po dokončení kroku nebo při změně stavu běhu.

2.5 Agentic Engineering (Agentické inženýrství)

V této práci zastřešuje návrh systémů, které kolem jazykového modelu zajišťují nástroje, kontext, stav, provádění a kontrolní mechanismy.

Předmětem není trénování modelu, ale inženýrství systému, který z modelových rozhodnutí vytváří řízené akce nad prostředím. ^(18, 37)

2.5.1 Guardrail (Deterministický mantinel) [Deterministic Guardrail]

Programově vynucená kontrola, která může před pokračováním běhu validovat nebo zablokovat vstup, výstup či použití nástroje. Guardrails lze spouštět na hranicích vstupu, výstupu nebo volání nástroje a při nesplnění podmínky běh zastavit či odmítnout konkrétní akci. ⁽³⁸⁾

2.5.2 HITL (Zapojení člověka do smyčky) [Human-in-the-loop]

Uspořádání automatizovaného procesu, ve kterém člověk v určených bodech poskytuje zpětnou vazbu, schválení nebo rozhodnutí. HITL ponechává část rozhodovacího procesu člověku místo úplné automatizace a používá lidský zásah jako součást řízení nebo kontroly systému. ⁽³⁹⁾

2.5.3 Sandbox (Izolované běhové prostředí)

Omezené běhové prostředí, které odděluje prováděný kód a jeho oprávnění od hostitelského systému. Izolační hranice mohou omezovat přístup k systémovým prostředkům; microVM představují jednu z možností silnější izolace nedůvěryhodných workloadů. ⁽⁴⁰⁾

2.5.4 Harness Engineering (Harnessové inženýrství)

Návrh a iterativní úprava harnessu s cílem řídit chování agenta, jeho pracovní stav, provádění a ověřování. Konkrétní struktura harnessu ovlivňuje výkon dlouhotrvajících agentních úloh a musí se přizpůsobovat schopnostem používaného modelu místo hromadění nepotřebného scaffolding. ⁽⁴¹⁾

2.5.5 Prompt Engineering (Promptové inženýrství)

Systematický návrh instrukcí, příkladů a jejich struktury s cílem ovlivnit chování jazykového modelu. ⁽⁴²⁾ Prompt řídí pravděpodobnostní chování modelu; pravidla, která musí systém technicky vy-
nutit, patří do **Guardrail (Deterministický mantinel) [Deterministic Guardrail]** nebo jiné běhové kontroly.

System Prompt (Systémový prompt)

Systémová instrukční vrstva, která vymezuje roli, pravidla a výchozí způsob chování modelu nebo agenta. Systémový prompt poskytuje stabilní instrukční kontext, ale sám o sobě není technickou izolační ani autorizační hranicí. ⁽⁴²⁾

2.5.6 Loop Engineering (Inženýrství prováděcí smyčky) [Execution-loop Engineering]

V této práci označuje návrh pravidel, která řídí opakování agentní smyčky, její stavové přechody a podmínky ukončení.

Řízení smyčky může zahrnovat rozpočty, opakování selhaných kroků, ukončení neproduktivního běhu nebo předání řízení jiné fázi systému. ⁽⁴¹⁾

2.5.7 Graph Engineering (Inženýrství pracovních grafů) [Workflow-graph Engineering]

V této práci označuje návrh vícefázových agentních workflow jako explicitních uzlů, závislostí a přechodů.

Grafová struktura umožňuje oddělit role a fáze, vyjádřit jejich závislosti a řídit, které části práce mohou probíhat sekvenčně nebo paralelně. Multiagentní orchestrace využívá obdobné dělení rolí a komunikace mezi agenty. ⁽²⁹⁾

DAG (Orientovaný acyklický graf) [Directed Acyclic Graph]

Orientovaný graf bez orientovaného cyklu používaný k vyjádření závislostí mezi kroky workflow.

V pracovním grafu představují uzly dílčí kroky a hrany jejich závislosti; acykličnost umožňuje určit pořadí provedení bez návratu do předchozího uzlu.

2.5.8 Context Engineering (Kontextové inženýrství)

Systematický výběr a správa informací, které jsou modelu zpřístupněny v aktivním kontextu během inference. ⁽⁴³⁾

Kontext zahrnuje nejen prompt, ale také nástroje, externí data a historii zpráv; jeho obsah je nutné průběžně kurátorovat vzhledem k omezené kapacitě a nerovnoměrnému využití dlouhého kontextu. ^(21, 43)

Context Injection (Vkládání kontextu)

Cílené vložení relevantních informací do aktivního kontextu až v okamžiku, kdy jsou potřebné pro aktuální krok.

Just-in-time přístup umožňuje udržovat v kontextu pouze odkazy nebo minimální stav a potřebná data načíst nástrojem až během běhu. ⁽⁴³⁾

Compaction (Kompakce kontextu) [Context Compaction]

Zmenšení aktivního kontextu nahrazením části historie kratší reprezentací, typicky shrnutím nebo výběrem důležitých informací. ⁽⁴³⁾

Kompakce uvolňuje kapacitu pro další běh, ale příliš agresivní komprese může odstranit detaily, které se později ukážou jako důležité. ^(43, 44)

RAG (Generování rozšířené vyhledávání) [Retrieval-Augmented Generation]

Architektura, ve které systém před generováním vyhledá relevantní informace z externího zdroje a poskytne je modelu jako další kontext. ⁽⁴⁵⁾

RAG odděluje znalost uloženou v externím korpusu od parametrů modelu a umožňuje vybírat podklady podle aktuálního dotazu. ⁽⁴⁵⁾

Sémantický posun

V této práci označuje postupné zkreslení pracovního významu při opakovaném ztrátovém shrnování nebo transformaci kontextu.

Kompakce nutně vybírá, co zachovat a co vypustit; opakované komprese proto mohou odstranit jemné, ale později důležité informace a změnit pracovní reprezentaci původního stavu. ^(43, 44)

3 Praktická část

Aplikace teoretických principů na konkrétní systém DarkFactory. Praktická část popisuje ověřenou architekturu DarkFactory a hodnotí doložitelné vlastnosti výsledného řešení.

3.1 Úvod

Vymezení způsobu, jakým praktická část aplikuje teoretické mechanismy na systém DarkFactory.

Praktická část popisuje pouze ověřenou implementaci DarkFactory a následně hodnotí chování a důkazy, které lze skutečně doložit.

3.2 Architektura DarkFactory

Konkrétní uspořádání komponent agentního harnessu do jednoho vývojového systému. Spojuje modelovou, stavovou, nástrojovou a kontrolní vrstvu do řízeného toku provádění softwarových úloh.

 **Hlubková kritika / Oponentura:** Tato část zatím popisuje architekturu pouze obecně. Před finalizací doplnit skutečné komponenty DarkFactory, jejich rozhraní, tok stavu a ověřené vazby na implementaci; nevymýšlet je pouze z návrhového záměru.

3.3 Výsledky a diskuse

Vyhodnocení, jak návrh DarkFactory odpovídá stanoveným cílům a výzkumným otázkám.

Výsledky odpovídají na O1–O3 a odděleně vymezují hranice toho, co lze z provedeného ověření tvrdit.

3.3.1 Vyhodnocení výzkumných otázek

Posouzení, jak navržená architektura DarkFactory odpovídá na stanovené otázky.

O1 — autonomie při zachování lidského dohledu: harness odděluje zadání, provádění, automatické kontroly a lidská rozhodnutí tak, aby rutinní kroky mohl vykonávat agent a člověk zůstal u významných rozhodnutí.

O2 — rozpoznání neproduktivního běhu: návrh používá explicitní stav, limity běhu, deterministické kontroly a eskalaci k člověku namísto neomezeného pokračování modelové smyčky.

O3 — zachování kontextu: trvalý stav úlohy je oddělen od omezeného kontextového okna modelu a potřebné informace lze průběžně komprimovat a znovu načítat.

 **Chyba / Nesrovnalost k opravě:** Každou odpověď O1–O3 propojit s konkrétním ověřitelným prvkem návrhu nebo implementace a s výsledkem kontroly/testu. Současné znění je argumentační shrnutí, nikoli ještě doložená evaluace.

3.3.2 Omezení evaluace

Vymezení tvrzení, která nelze z architektonického a implementačního ověření spolehlivě odvodit.

Práce nepředstavuje statistický benchmark úspěšnosti agentů, ceny, latence ani četnosti zacyklení na reprezentativním souboru úloh. Taková tvrzení vyžadují samostatný kontrolovaný experiment.

Výsledky proto hodnotí návrh a jeho technické ověření, nikoli obecnou výkonnostní převahu DarkFactory nad jinými agentními systémy.

4 Závěr

Práce zkoumala, jak lze současnou agentní AI účinně zapojit do vývoje softwaru a jakou architekturu musí mít agent harness, aby spojoval autonomii s kontrolovatelným prováděním.

K O1 architektura přesouvá rutinní kroky na agentní systém a ponechává člověku zadání, kontrolu a důležitá rozhodnutí. K O2 používá explicitní stav, limity, kontrolní uzly a eskalaci namísto neomezené smyčky. K O3 odděluje trvalý stav úlohy od omezeného pracovního kontextu modelu.

DarkFactory tyto principy spojuje v konkrétní architektuře agentního harnessu.

Seznam obrázků a tabulek

Obrázek 1	Odhad používání AI ve světové populaci. Zdroj: CPA.RIP / Gradually AI ⁽³⁾	10
Obrázek 2	Původní tweet Andreje Karpathyho o Vibe Coding. ^(5, 7)	13
Obrázek 3	Ilustrativní 2D projekce vztahu král – muž + žena $\approx$ královna; osy Pohlaví a Královský status slouží pouze k názornému vysvětlení a nepředstavují doslov- né naučené dimenze embeddingového prostoru.	15
Obrázek 4	Aplikace Codex. Zdroj: OpenAI.	17
Obrázek 5	Claude Code při práci se souborem. Zdroj: Anthropic.	17
Obrázek 6	Aplikační rozhraní Claude. Zdroj: Anthropic.	18
Obrázek 7	Smyčka ReAct: model zvolí akci, harness ji provede a výsledek vrátí modelu. .	19

Seznam příloh

Seznam zdrojů

1. OPENAI. Introducing the Codex app. Online. 2026. [Accessed 20 září 2026]. Available from: <https://openai.com/index/introducing-the-codex-app/>
2. ANTHROPIC. Claude Code. Online. 2026. [Accessed 20 září 2026]. Available from: <https://claude.com/product/claude-code>
3. CPA.RIP. How Many People Worldwide Use Generative AI. Online. 2026. [Accessed 21 září 2026]. Available from: <https://cpa.rip/en/news/ai-usage-worldwide/>
4. SOMMERVILLE, Ian. *Software Engineering*. 10. Pearson, 2016. ISBN 978-0-13-394303-0.
5. KARPATHY, Andrej. A new kind of coding I call vibe coding. Online. 2025. [Accessed 20 září 2026]. Available from: <https://twitter.com/karpathy/status/1886192184808149383>
6. WILLISON, Simon. A quote from Andrej Karpathy. Online. 2025. [Accessed 20 září 2026]. Available from: <https://simonwillison.net/2025/Feb/6/andrej-karpathy/>
7. KRAVETS, David. A semantic history: How the term 'vibe coding' went from a tweet to prod. Online. 2026. [Accessed 21 září 2026]. Available from: <https://www.coderabbit.ai/blog/a-semantic-history-how-the-term-vibe-coding-went-from-a-tweet-to-prod>
8. CAMBRIDGE UNIVERSITY PRESS. AI slop. Online. 2026. [Accessed 20 září 2026]. Available from: <https://dictionary.cambridge.org/dictionary/english/ai-slop>
9. BÖCKELER, Birgitta. Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessel. Online. 2025. [Accessed 21 září 2026]. Available from: <https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html>
10. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. ISBN 978-1-4842-0076-6.
11. GITHUB. Branches | GitHub Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://docs.github.com/en/pull-requests/reference/branches>
12. GITHUB. About pull requests | GitHub Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://docs.github.com/en/pull-requests/get-started/about-pull-requests>
13. MERKEL, Dirk. Docker: lightweight linux containers for consistent development and deployment. *Linux Journal*. 2014. Vol. 2014, no. 239, p. 2.
14. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
15. GITHUB. Understanding GitHub Actions. Online. 2026. [Accessed 21 září 2026]. Available from: <https://docs.github.com/en/actions/get-started/understand-github-actions>

16. DOCKER. What is a container?. Online. 2026. [Accessed 21 září 2026]. Available from: <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>
17. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
18. ANTHROPIC. Building effective agents. Online. 2024. Available from: <https://www.anthropic.com/research/building-effective-agents>
19. SENNRICH, Rico, HADDOW, Barry a BIRCH, Alexandra. Neural Machine Translation of Rare Words with Subword Units. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Online. 2016. P. 1715–1725. Available from: <https://aclanthology.org/P16-1162/>
20. MIKOLOV, Tomas, CHEN, Kai, CORRADO, Greg a DEAN, Jeffrey. Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781*. Online. 2013. Available from: <https://arxiv.org/abs/1301.3781>
21. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
22. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
23. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>
24. ANTHROPIC. Navigating the Claude desktop app. Online. 2026. [Accessed 20 září 2026]. Available from: <https://academy.claude.com/tutorials/navigating-the-claude-desktop-app>
25. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
26. OPENAI. Sessions | OpenAI Agents SDK. Online. 2026. [Accessed 21 září 2026]. Available from: <https://openai.github.io/openai-agents-js/guides/sessions/>

27. SHINN, Noah, CASSANO, Federico, GOPINATH, Ashwin, NARASIMHAN, Karthik a YAO, Shunyu. Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems*. Online. 2023. Vol. 36, p. 8634–8652. Available from: <https://arxiv.org/abs/2303.11366>
28. ANTHROPIC. Create plugins | Claude Code Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://code.claude.com/docs/en/plugins>
29. WU, Qingyun, BANSAL, Gagan, ZHANG, Jieyu, WU, Yiran, LI, Beibin, ZHU, Erkang, JIANG, Li, ZHANG, Xiaoyun, ZHANG, Chi, LIU, Jue, AWADALLAH, Ahmed Hassan, WHITE, Ryan W., BURGER, Doug a WANG, Chi. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*. Online. 2023. Available from: <https://arxiv.org/abs/2308.08155>
30. SCHICK, Timo, DWIVEDI-YU, Jane, DESSÌ, Roberto, RAILEANU, Roberta, LOMELI, Maria, ZETTMAYER, Luke, CANCELLA, Nicola a SCIALOM, Thomas. Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems*. Online. 2023. Vol. 36, p. 68539–68551. Available from: <https://arxiv.org/abs/2302.04761>
31. WRIGHT, Austin, ANDREWS, Henry, HUTTON, Ben a DENNIS, Greg. JSON Schema Draft 2020-12. Online. 2022. [Accessed 20 září 2026]. Available from: <https://json-schema.org/draft/2020-12>
32. OPENAI. Introducing Structured Outputs in the API. Online. 2024. [Accessed 20 září 2026]. Available from: <https://openai.com/index/introducing-structured-outputs-in-the-api/>
33. ANTHROPIC. Code execution tool. Online. 2026. [Accessed 20 září 2026]. Available from: <https://platform.claude.com/docs/en/agents-and-tools/tool-use/code-execution-tool>
34. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>
35. ANTHROPIC. Connect Claude Code to tools via MCP. Online. 2026. [Accessed 21 září 2026]. Available from: <https://code.claude.com/docs/en/mcp>
36. AGENT SKILLS. Agent Skills Specification. Online. 2026. [Accessed 20 září 2026]. Available from: <https://agentskills.io/specification>
37. WANG, Lei, MA, Chen, FENG, Xueyang, ZHANG, Zeyu, YANG, Hao, ZHANG, Jingsen, CHEN, Zhiyuan, TANG, Jiakai, CHEN, Xu, LIN, Yankai, ZHAO, Wayne Xin, WEI, Zhewei a WEN, Ji-Rong. A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science*. Online. 2024. Vol. 18, no. 6, p. 186345. Available from: <https://arxiv.org/abs/2308.11432>

38. OPENAI. Guardrails | OpenAI Agents SDK. Online. 2026. [Accessed 21 září 2026]. Available from: <https://openai.github.io/openai-agents-js/guides/guardrails/>
39. MOSQUEIRA-REY, Eduardo, HERNÁNDEZ-PEREIRA, Elena, ALONSO-RÍOS, David, BOBES-BASCARÁN, José a FERNÁNDEZ-LEAL, Ángel. Human-in-the-loop machine learning: a state of the art. *Artificial Intelligence Review*. Online. 2023. Vol. 56, no. 4, p. 3005–3054. Available from: <https://doi.org/10.1007/s10462-022-10246-w>
40. AGACHE, Alexandru, DEACONESCU, Razvan, IORDACHE, Mihai, LUPU, Alexandra, RADU, Vlad, BUGA, Cezar, BALAN, Catalin a FLORESCU, Andreea. Firecracker: Lightweight Virtualization for Serverless Applications. In : *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. Online. 2020. p. 419–434. Available from: <https://www.usenix.org/conference/nsdi20/presentation/agache>
41. ANTHROPIC. Harness design for long-running application development. Online. 2026. [Accessed 21 září 2026]. Available from: <https://www.anthropic.com/engineering/harness-design-long-running-apps>
42. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
43. ANTHROPIC. Effective context engineering for AI agents. Online. 2025. [Accessed 21 září 2026]. Available from: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
44. JIANG, Huiqiang, WU, Qianhui, LIN, Chin-Yew, YANG, Yuqing a QIU, Lili. LLMingua: Compressing Context for Accelerated Inference of Large Language Models. *arXiv preprint arXiv:2310.05736*. Online. 2023. Available from: <https://arxiv.org/abs/2310.05736>
45. LEWIS, Patrick, PEREZ, Ethan, PIKTUS, Aleksandra, PETRONI, Fabio, KARPUKHIN, Vladimir, GOYAL, Naman, KÜTTLER, Heinrich, LEWIS, Mike, YIH, Wen-tau, ROCKTÄSCHEL, Tim, RIEDEL, Sebastian a KIELA, Douwe. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*. Online. 2020. Vol. 33, p. 9459–9474. Available from: <https://arxiv.org/abs/2005.11401>
46. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
47. SHI, Yifan, ZHANG, Wei a CUI, Tianyi. A Programming Paradigm for Spatiotemporal Composability. *arXiv preprint arXiv:2608.25512*. Online. 2026. Available from: <https://arxiv.org/abs/2608.25512>
48. WOOLDRIDGE, Michael a JENNINGS, Nicholas R. Intelligent Agents: Theory and Practice. *The Knowledge Engineering Review*. 1995. Vol. 10, no. 2, p. 115–152.
49. KINSMAN, Timothy, WESSEL, Mairieli, GEROSA, Marco A. a TREUDE, Christoph. How do software developers use GitHub Actions to automate their workflows?. In : *Procee-*

- dings of the 18th International Conference on Mining Software Repositories*. 2021. p. 420–431.
50. DABBISH, Laura, STUART, Colleen, TSAY, Jason a HERBSLEB, Jim. Social coding in GitHub: transparency and collaboration in an open software repository. In : *Proceedings of the ACM 2012 conference on Computer Supported Cooperative Work*. 2012. p. 1277–1286.
 51. OPENAI. The ChatGPT home page. Online. 2026. [Accessed 20 září 2026]. Available from: <https://help.openai.com/en/articles/9125172>
 52. OPENAI DEVELOPER COMMUNITY. Seeking Community Advice: Best Practices for Creating a Comprehensive GPT Manual. Online. 2024. [Accessed 20 září 2026]. Available from: <https://community.openai.com/t/seeking-community-advice-best-practices-for-creating-a-comprehensive-gpt-manual/1033512>
 53. FOWLER, Martin. Vibe Coding. Online. 2026. [Accessed 20 září 2026]. Available from: <https://martinfowler.com/bliki/VibeCoding.html>
 54. GONÇALVES, Pavlína Wurzel, FREGNAN, Enrico, BAUM, Tobias, SCHNEIDER, Kurt a BACCHELLI, Alberto. Do explicit review strategies improve code review performance? Towards understanding the role of cognitive load. *Empirical Software Engineering*. Online. 2022. Vol. 27, no. 4, p. 99. DOI [10.1007/s10664-022-10123-8](https://doi.org/10.1007/s10664-022-10123-8).
 55. GVISOR AUTHORS. Introduction to gVisor security. Online. 2026. [Accessed 20 září 2026]. Available from: https://gvisor.dev/docs/architecture_guide/intro/
 56. MARIUS, Patrik. DarkFactory. Online. 2026. [Accessed 20 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
 57. ANTHROPIC. Hooks reference | Claude Code Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://code.claude.com/docs/en/hooks>
 58. ANTHROPIC. Scaling Managed Agents: Decoupling the brain from the hands. Online. 2026. [Accessed 21 září 2026]. Available from: <https://www.anthropic.com/engineering/managed-agents>